Praktikum 8 – Siklus Pengambilan

NRP : 1. 3223600038 . . . . . .

2. 3223600041 . . . . . .

3. 3223600048 . . . . . .

4. 3223600050. . . . . .

5. 3223600058. . . . . .

Nama : 1. Karolus Wira Wicaksana..... . . . . .

2. Akmal Rafiuddin..... . . . . .

3. Moch. Faris Afif..... . . . . .

4. Labib Al Afaf.. . . . .

5. Ihsanta Zaki Sanjaya..... . . . . .

: 2 D4 Teknik Komputer B. . . . . .

: Praktikum Desain dadn Organisasi Komputer . . . . . .

D4 TEKNIK KOMPUTER

POLITEKNIK ELEKTRONIKA NEGERI SURABAYA

2025

BAB 1 TUJUAN

a. Mahasiswa dapat menjelaskan dan membuat program siklus pengambilan.
b. Mahasiswa dapat menjelaskan dan membuat program keadaan Alamat.
c. Mahasiswa dapat menjelaskan dan membuat program keadaan penembahan.
d. Mahasiswa dapat menjelaskan dan membuat program keadaan memori.

- **BAB 2 DASAR TEORI**

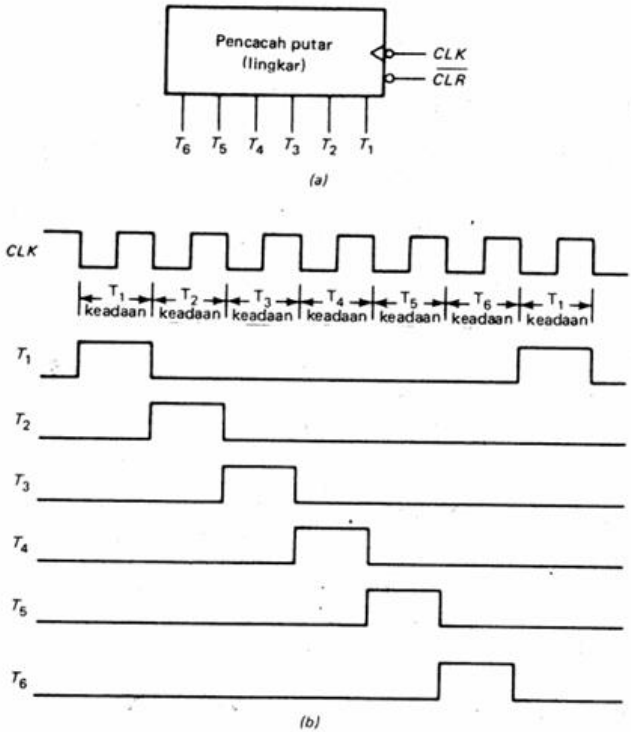
Unit kendali adalah kunci dari pengoperasian komputer secara otomatik. Unit kendali membangkitkan atau mengeluarkan kata-kata kendali untuk mengambil dan melaksanakan setiap instruksi. Pada waktu suatu instruksi diambil dan dilaksanakan, komputer akan melewati beberapa keadaan pewaktuan (timming state: disingkat T state = keadaan T), yaitu periode-periode waktu pada saat mana terjadi perubahan isi-isi register. Berikut ini akan kita pelajari lebih lanjut arti keadaan T tersebut.

Gambar 2a melukiskan simbol pencacah lingkar dengan keluaran.

$T = T_6T_5T_4T_3T_2T_1$

Pada permulaan operasi komputer kata lingkar T menunjukkan pulsa detak yang berturut turut menghasilkan kata-kata lingkar :

T = 000010
T = 000100
T = 001000
T = 010000
T = 100000



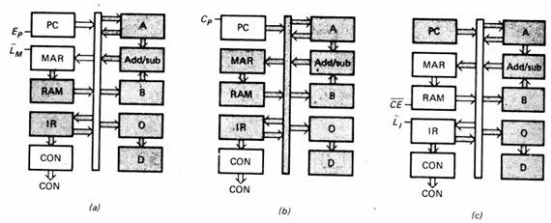
Kemudian pencacah lingkar direset menjadi 000001, dan siklus yang sama akan berulang. Setiap kata lingkar merepresentasikan satu keadaan T.

Gambar 2b memperlihatkan pulsa-pulsa pewaktuan yang dikeluarkan oleh pencacah lingkar. Keadaan T_1 berawal pada suatu tepi negatif pulsa detak dan berakhir pada tepi negatif yang berikutnya. Selama keadaan T ini, bit keluarkan T_1 dari pencacah lingkar merupakan tingkat logika tinggi.

Keadaan-keadaan Y yang selanjutnya berturut-turut T_2 tinggi, disusul T_3 tinggi, kemudian T_4 tinggi, dan seterusnya. Sebagaimana dapat kita lihat, pencacah putar menghasilkan enam keadaan T. Setiap instruksi diambil dan dilaksanakan selama enam keadaan T ini. Perhatikan bahwa tepi CLK yang positif terjadi dipertengahan setiap keadaan T.

A. Keadaan Alamat

Keadaan Alamat Keadaan T_1 disebut keadaan alamat (address State) karena alamat didalam pencacah program (PC) dipindahkan kepada register alamat memori (MAR) selama keadaan ini. Gambar 3A memperlihatkan bagian-bagian komputer yang aktif selama keadaan T_1 (bagian aktif digambarkan terang, dan bagian tidak aktif gelap).



Selama keadaan alamat E_p dan L_m merupakan bit-bit yang aktif, sedangkan semua bit kendali yang lain tidak aktif. Ini berarti selama keadaan bagian pengendali pengurut (CON) mengeluarkan kata kendali berbentuk seperti dibawah ini.

$$\begin{aligned} CON &= C_p\, E_p\, \overline{L}_M\, \overline{CE} \quad \overline{L}_I\, \overline{E}_I\, \overline{L}_A\, E_A \quad S_u\, E_u\, \overline{L}_B\, \overline{L}_o \\ &= 0\ 1\ \ 0\ \ 1 \quad\ \ 1\ \ 1\ \ 0\ \ 0 \quad\ \ 0\ 0\ 1\ 1 \end{aligned}$$

B. Keadaan Penambahan

Gambar 3b memperlihatkan bagian-bagian aktif dari SAP-1 selama keadaan T2. Keadaan ini disebut keadaan penambahan (increment state) karena hitungan pada pencacah program ditingkatkan (ditambah) selama periode ini. Selama keadaan penambahan bagian pengendali-pengurut menghasilkan sebuah kata-kendali berbentuk :

$$\begin{aligned} CON &= C_p\, E_p\, \overline{L}_M\, \overline{CE} \quad \overline{L}_I\, \overline{E}_I\, \overline{L}_A\, E_A \quad S_u\, E_u\, \overline{L}_B\, \overline{L}_o \\ &= 1\ 0\ \ 1\ \ 1 \quad\ \ 1\ \ 1\ \ 1\ \ 0 \quad\ \ 0\ 0\ 1\ 1 \end{aligned}$$

Terlihat disini bahwa CP adalah bit yang aktif,

C. Keadaan Memori

Keadaan T_3 disebut keadaan memori (memory state) karena instruksi pada RAM dengan alamat yang ditunjuk dipindahkan dari memori ke register instruksi. Gambar 3c melukiskan bagian-bagian aktif dari SAP-1 selama keadaan memori ini. Selama keadaan ini bit-bit kendali yang aktif

hanyalah CE dan L1, dan kata yang dikeluarkan oleh bagian pengendali pengurut adalah

$$CON = C_p E_p \overline{L}_M \overline{CE} \quad \overline{L}_1 \overline{E}_1 \overline{L}_A E_A \quad S_u E_u \overline{L}_B \overline{L}_o$$

$$= 0\ 0\ 1\ 0 \quad 0\ 1\ 1\ 0 \quad 0\ 0\ 1\ 1$$

D. Siklus Pengambilan

Keadaan-keadaan alamat, penambahan, dan memori disebut siklus pengambilan/ penjemputan (Fetch cycle) dari SAP-1. Selama keadaan alamat, E_P dan L_M Maktif ini berarti pencacah program mengaktifkan MAR melalui bus W. Dalam gambar 2b, tepi positif sinyal detak muncul pada pertengahan keadaan alamat, peristiwa ini digunakan untuk mengisi MAR dengan isi PC.

C_P adalah satu-satunya bit kendali yang aktif selama keadaan penambahan. Pencacah program diaktifkan guna mencacah tepi positif pulsa detak. Dipertengahan keadaan penambahan, tepi positif sinyal detak diterima pencacah program dan ini meningkatkan cacahan dengan 1 angka .

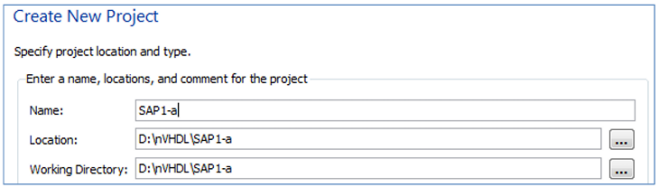
Dalam keadaan memori, bit-bit C_E dan L_1 yang aktif. Dengan ini, kata RAM dengan alamat yang ditentukan mengaktifkan register instruksi bus W. Pada pertengahan keadaan memori, sebuah tepi positif sinyal detak akan mengisi register instruksi dengan kata RAM tadi.

- **BAB 3 PERCOBAAN**

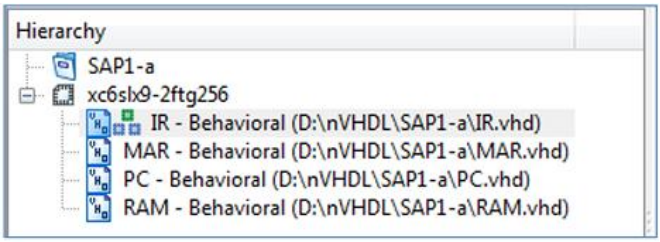
Membuat program siklus pengambilan

a. Membuat proyek SAP1-a top modul, program ini menggabungkan sub program PC, MAR, RAM dan IR.

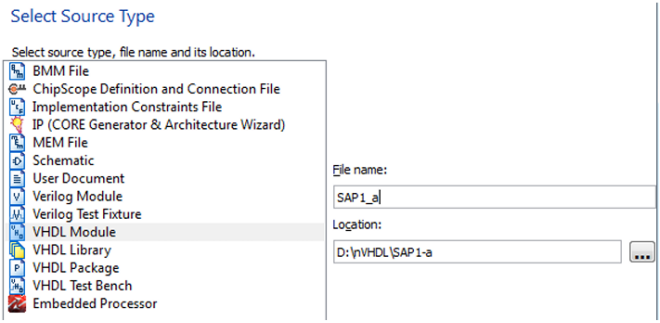
b. Buatlah proyek baru dengan nama SAP1-a.



c. Tambahkan program PC, MAR, RAM dan IR.



d. Tambahkan Source program baru dan beri nama SAP1_a

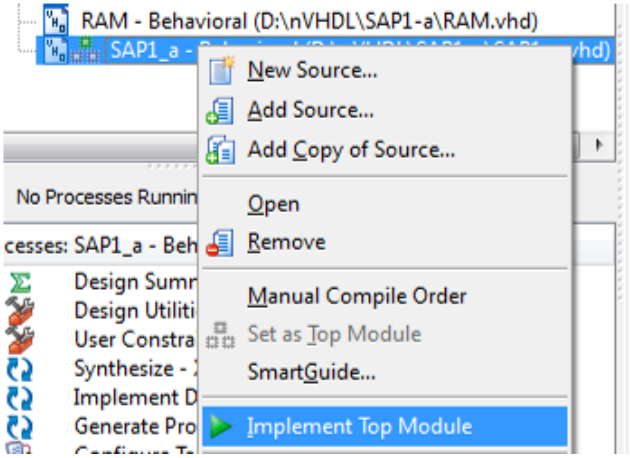


e. Tuliskan program SAP1_a seperti skrip dibawah ini.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity SAP1_a is
    Port ( clk : in  STD_LOGIC;
           clr : in  STD_LOGIC;
           Cp     : in  STD_LOGIC;
           Ep     : in  STD_LOGIC;
           nLm : in  STD_LOGIC;
           nCe : in  STD_LOGIC;
           nLi : in  STD_LOGIC;
           nEi : in  STD_LOGIC;
           wBus : out  STD_LOGIC_VECTOR (7 downto 0);
           IRtoCU : out  STD_LOGIC_VECTOR (3 downto 0)
        );
end SAP1_a;

architecture Behavioral of SAP1_a is
      component PC is
            Port ( nCLK : in  STD_LOGIC;
                   nCLR : in  STD_LOGIC;
                   Cp   : in  STD_LOGIC;
                   Ep   : in  STD_LOGIC;
                   BUS_Low : out  STD_LOGIC_VECTOR (3 downto 0));
      end component;
      component MAR is
            Port ( CLK : in  STD_LOGIC;
                   nLm : in  STD_LOGIC;
                   inMAR : in  STD_LOGIC_VECTOR (3 downto 0);
                   outMAR : out  STD_LOGIC_VECTOR (3 downto 0));
      end component;
      component RAM is
            Port ( nCe : in  STD_LOGIC;
                   inRAM : in  STD_LOGIC_VECTOR (3 downto 0);
                   outRAM : out  STD_LOGIC_VECTOR (7 downto 0));
      end component;
      component IR  is
            Port ( CLK : in  STD_LOGIC;
                   CLR : in  STD_LOGIC;
                   nLi : in  STD_LOGIC;
                   nEI : in  STD_LOGIC;
                   inIR : in  STD_LOGIC_VECTOR (7 downto 0);
                   outIRlow : out  STD_LOGIC_VECTOR (3 downto 0);
                   outIRhigh : out  STD_LOGIC_VECTOR (3 downto
0));
      end component;

      signal sig_bus : STD_LOGIC_VECTOR (7 downto 0):=x"00";
      signal sig_MARtoRAM : STD_LOGIC_VECTOR (3 downto 0):=x"0";

begin
      u1: PC port map(
                  nCLK  => CLK,
                  nCLR  => CLR,
                  Cp    => Cp,
                  Ep    => Ep,
                  BUS_Low => sig_bus(3 downto 0)      );
      u2: MAR port map(
                  CLK   => CLK,
                  nLm   => nLm,
                  inMAR =>  sig_bus(3 downto 0),
                  outMAR => sig_MARtoRAM                    );
      u3: RAM port map(
                  nCe   => nCe,
                  inRAM => sig_MARtoRAM,
```

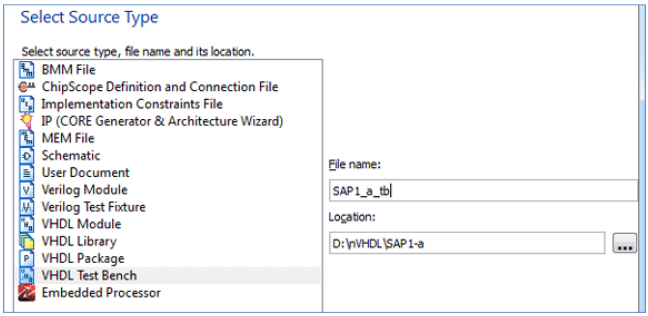
```
                    outRAM => sig_bus
        );
        u4: IR port map(
                    CLK    => CLK,
                    CLR    => CLR,
                    nLi    => nLi,
                    nEi    => nEi,
                    inIR   => sig_bus,
                    outIRlow  => sig_bus(3 downto 0),
                    outIRhigh => IRtoCU      );
        wBus <= sig_bus;
end Behavioral;
```

f. Atur agar SAP1_a menjadi top module.



g. Lakukan Systhesize-XST dan pastikan tidak ada error.

h. Tambahkan program untuk simulasinya, beri nama file SAP1_a_tb

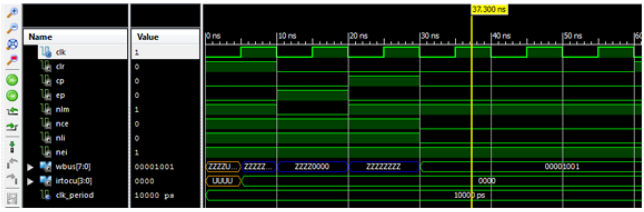


i. Tuliskan progran SAP1_a_tb, seperti pada skrip dibawah ini.

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY SAP1_a_tb IS
END SAP1_a_tb;

ARCHITECTURE behavior OF SAP1_a_tb IS

    COMPONENT SAP1_a
    PORT(
         clk : IN  std_logic;
         clr : IN  std_logic;
         Cp : IN  std_logic;
         Ep : IN  std_logic;
         nLm : IN  std_logic;
         nCe : IN  std_logic;
         nLi : IN  std_logic;
         nEi : IN  std_logic;
         wBus : OUT  std_logic_vector(7 downto 0);
         IRtoCU : OUT  std_logic_vector(3 downto 0)
        );
    END COMPONENT;

   --Inputs
   signal clk    : std_logic := '0';
   signal clr    : std_logic := '0';
   signal Cp     : std_logic := '0';
   signal Ep     : std_logic := '0';
   signal nLm    : std_logic := '1';
   signal nCe    : std_logic := '1';
   signal nLi    : std_logic := '1';
   signal nEi    : std_logic := '1';

     --Outputs
   signal wBus : std_logic_vector(7 downto 0):=x"00";
   signal IRtoCU : std_logic_vector(3 downto 0):=x"0";

   -- Clock period definitions
   constant clk_period : time := 10 ns;

BEGIN

     -- Instantiate the Unit Under Test (UUT)
   uut: SAP1_a PORT MAP (
          clk => clk,
          clr => clr,
          Cp => Cp,
          Ep => Ep,
          nLm => nLm,
          nCe => nCe,
          nLi => nLi,
          nEi => nEi,
          wBus => wBus,
          IRtoCU => IRtoCU
         );
```

```
   -- Clock process definitions
   clk process :process
   begin
           clk <= '0';        wait for clk_period/2;
           clk <= '1';        wait for clk_period/2;
   end process;

   -- Stimulus process
   stim proc: process
   begin
      clr <= '1';             wait for clk period;
           clr <= '0';        --wait for clk_period;
 -- T1
           Cp  <= '0';        Ep  <= '1';
           nLm <= '0';
           nCe <= '1';
           nLi <= '1';     nEi <= '1';
           wait for CLK_period;
--    T2
           Cp  <= '1';        Ep  <= '0';
           nLm <= '1';
           nCe <= '1';
           nLi <= '1';     nEi <= '1';
           wait for CLK_period;
-- T3
           Cp  <= '0';        Ep  <= '0';
           nLm <= '1';
           nCe <= '0';
           nLi <= '0';     nEi <= '1';

           wait for clk_period*3;
   end process;
END;
```

j. Tampilkan hasil timing diagramnya. Seperti pada gambar dibawah ini.



- **BAB 4 PROGRAM DAN HASIL**

Program SAP1_a (vhdl module)

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity SAP1_a is
   Port (
      clk     : in  STD_LOGIC;
      clr     : in  STD_LOGIC;
      Cp      : in  STD_LOGIC;
      Ep      : in  STD_LOGIC;
      nLm     : in  STD_LOGIC;
      nCe     : in  STD_LOGIC;
      nLi     : in  STD_LOGIC;
      nEi     : in  STD_LOGIC;
      wBus    : out STD_LOGIC_VECTOR (7 downto
0);
      IRtoCU     :   out   STD_LOGIC_VECTOR   (3
downto 0)
   );
end SAP1_a;

architecture Behavioral of SAP1_a is

   component PC is
      Port (
         nCLK     : in  STD_LOGIC;
         nCLR     : in  STD_LOGIC;
         Cp       : in  STD_LOGIC;
         Ep       : in  STD_LOGIC;
         BUS_Low  : out STD_LOGIC_VECTOR (3
downto 0)
      );
   end component;

   component MAR is
      Port (
         CLK     : in  STD_LOGIC;
```

```
            nLm     : in  STD_LOGIC;
            inMAR     :  in    STD_LOGIC_VECTOR  (3
downto 0);
            outMAR    :  out  STD_LOGIC_VECTOR  (3
downto 0)
        );
    end component;

    component RAM is
        Port (
            nCe     : in  STD_LOGIC;
            inRAM     :  in    STD_LOGIC_VECTOR  (3
downto 0);
            outRAM    :  out  STD_LOGIC_VECTOR  (7
downto 0)
        );
    end component;

    component IR is
        Port (
            CLK         : in  STD_LOGIC;
            CLR         : in  STD_LOGIC;
            nLi         : in  STD_LOGIC;
            nEI         : in  STD_LOGIC;
            inIR            :  in    STD_LOGIC_VECTOR  (7
downto 0);
            outIRlow      : out STD_LOGIC_VECTOR (3
downto 0);
            outIRhigh     : out STD_LOGIC_VECTOR (3
downto 0)
        );
    end component;

    signal  sig_bus          : STD_LOGIC_VECTOR  (7
downto 0) := x"00";
    signal sig_MARtoRAM  : STD_LOGIC_VECTOR
(3 downto 0) := x"0";

begin
```

```
        u1: PC port map(
            nCLK     => CLK,
            nCLR     => CLR,
            Cp       => Cp,
            Ep       => Ep,
            BUS_Low  => sig_bus(3 downto 0)
        );

        u2: MAR port map(
            CLK      => CLK,
            nLm      => nLm,
            inMAR    => sig_bus(3 downto 0),
            outMAR   => sig_MARtoRAM
        );

        u3: RAM port map(
            nCe      => nCe,
            inRAM    => sig_MARtoRAM,
            outRAM   => sig_bus
        );

        u4: IR port map(
            CLK        => CLK,
            CLR        => CLR,
            nLi        => nLi,
            nEi        => nEi,
            inIR       => sig_bus,
            outIRlow   => sig_bus(3 downto 0),
            outIRhigh  => IRtoCU
        );

        wBus <= sig_bus;

end Behavioral;
```

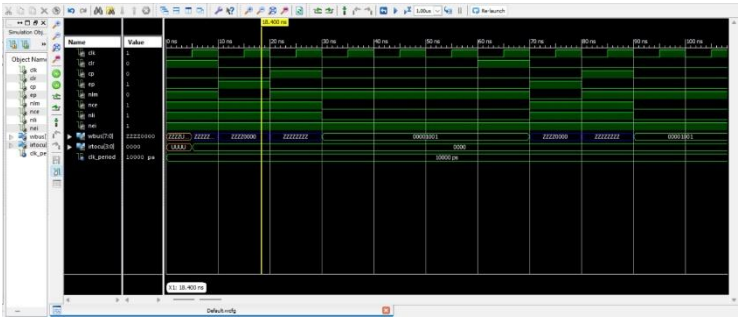
Program Test Bench

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY SAP1_a_tb IS
END SAP1_a_tb;

ARCHITECTURE behavior OF SAP1_a_tb IS

   COMPONENT SAP1_a
      PORT (
         clk     : IN  std_logic;
         clr     : IN  std_logic;
         Cp      : IN  std_logic;
         Ep      : IN  std_logic;
         nLm     : IN  std_logic;
         nCe     : IN  std_logic;
         nLi     : IN  std_logic;
         nEi     : IN  std_logic;
         wBus    : OUT std_logic_vector(7 downto
0);
         IRtoCU      :   OUT   std_logic_vector(3
downto 0)
      );
   END COMPONENT;

   -- Inputs
   signal clk     : std_logic := '0';
   signal clr     : std_logic := '0';
   signal Cp      : std_logic := '0';
   signal Ep      : std_logic := '0';
   signal nLm     : std_logic := '1';
   signal nCe     : std_logic := '1';
   signal nLi     : std_logic := '1';
   signal nEi     : std_logic := '1';
```

```
    -- Outputs
    signal wBus     : std_logic_vector(7 downto 0)
:= x"00";
    signal IRtoCU  : std_logic_vector(3 downto 0)
:= x"0";

    -- Clock period definition
    constant clk_period : time := 10 ns;

BEGIN

    -- Instantiate the Unit Under Test (UUT)
    uut: SAP1_a PORT MAP (
        clk     => clk,
        clr     => clr,
        Cp      => Cp,
        Ep      => Ep,
        nLm     => nLm,
        nCe     => nCe,
        nLi     => nLi,
        nEi     => nEi,
        wBus    => wBus,
        IRtoCU  => IRtoCU
    );

    -- Clock process definitions
    clk_process : process
    begin
        clk <= '0';
        wait for clk_period/2;
        clk <= '1';
        wait for clk_period/2;
    end process;
```

```
        -- Stimulus process
        stim_proc : process
        begin
            clr <= '1';
            wait for clk_period;
            clr <= '0';

            -- T1
            Cp  <= '0'; Ep  <= '1';
            nLm <= '0';
            nCe <= '1';
            nLi <= '1'; nEi <= '1';
            wait for clk_period;

            -- T2
            Cp  <= '1'; Ep  <= '0';
            nLm <= '1';
            nCe <= '1';
            nLi <= '1'; nEi <= '1';
            wait for clk_period;

            -- T3
            Cp  <= '0'; Ep  <= '0';
            nLm <= '1';
            nCe <= '0';
            nLi <= '0'; nEi <= '1';

            wait for clk_period * 3;
        end process;

END;
```

Hasil



- **BAB 5 ANALISA**

  Fetch Cycle adalah siklus pengambilan data ke memori atau register. Aliran data siklus pengambilan (Fetch Cycle) berdasarkan urutan kejadian selama siklus interuksi tergantung pada rangan CPU. Sebuah CPU yang menggunakan register memori alamat (MAR), Register memori buffer (MBR), pencacah program (PC) dan register instruksi(IR). Proses pada siklus pengambilan (Fetch Cycle), instruksi dibaca dari memori. Selanjutnya, PC berisi Alamat instruksi berikutnya yang akan diambil. Alamat ini dipindahkan di MAR(Memory Address Register) dan ditaruh di bus alamat. Kemudian Control Unit meminta pembacaan memori dan hasilnya disimpan di bus data dan disalin ke MBR (Memory Buffer Register), kemudian dipindahkan ke IR (Instruction Regiter). PC (Program Counter) naik nilainya 1, sebagai persiapan untuk pengambilan selanjutnya. Siklus selesai, CU

memeriksa isi IR (Instruction Register) untuk menentukan apakah IR (Instruction Register) berisi Operand Specifier yang menggunakan pengalamatan tak langsung (Indirect).

Dengan demikian siklus Fetch sederhana sebenarnya terdiri atas 3 langkah dan 4 operasi mikro. Secara simbolik dapat ditulis sebagai berikut:

T1 : MAR ← (PC)

T2 : MBR ← (MEMORY)

PC ← (PC)+1

T3 : IR ← (MBR)

Dimana T1, T2, dan T3 merupakan unit waktu yang berdurasi sama dan berurutan.

Pada praktikum ini yaitu membuat program siklus pengambilan SAP1. Dalam program ini menggunakan beberapa program lainnya seperti RAM, MAR (Memory Address Register), IR (Instruction Register), dan PC (Program Counter) untuk membangun sebuah program SAP1.

SAP1 adalah sebagai top modul. Program Siklus Pengambilan terdiri dari 3 keadaan yaitu alamat, penambahan, dan memori. Pada keadaan alamat yang aktif hanya PC, MAR, dan CON. Selama keadaan Alamat Ep dan Lm berbilai HIGH. Pada keadaan penambahan hanya Cp yang bernilai HIGH. Pada keadaan memori hanya terdapat CE dan L1 yan bernilai HIGH.

- **BAB 6 KESIMPULAN**

Fetch Cycle adalah siklus pengambilan data dari memori atau register dalam CPU. Siklus ini terdiri dari 3 langkah dan 4 operasi mikro yang bergantung pada rancangan CPU. Program Siklus Pengambilan memiliki 3 keadaan, yaitu alamat, penambahan, dan memori. Kode VHDL digunakan untuk membuat program keadaan alamat dan penambahan pada siklus pengambilan. Testbench dan Unit Under Test (UUT) digunakan untuk mensimulasikan dan menguji perilaku modul SAP1_a dalam merespon kondisi alamat dan penambahan pada siklus pengambilan.